Sprint 3 Review Date: April 10th
Scrum Master for Sprint 3:

Siddharth Mulupuru

Section 1 - Sprint 2 Retrospective → Sprint 3 Operational Plan

Sprint 2 Structural Issue:

The team lacked transparency around task progress, making it hard to know what was done and what was still being worked on.

Sprint 2 Rule (exact wording):

All team members must keep the GitHub board updated by moving tasks to in progress and done as work happens.

What this will look like during daily work:

All team members who complete a given task during class or outside of class will immediately update its status and every day we will include a task board checkin to see if all items are updated.

Who is responsible for enforcement:

Varun Hari

How will this be enforced? (select all that apply):

☒Daily Scrum checks

☒Board update checks

☐Task size enforcement

☐PR workflow enforcement

☐Other: Click or tap here to enter text.

When will enforcement occur during the Sprint?

As mentioned previously, at the start of each scrum, we will do a task board check to see if everything is updated. If something appears not to be updated, it's status will immediately be changed.

Section 2 – Sprint Goal (MVP Completion + Stabilization)

Sprint Goal (By the end of Sprint 3, a user will be able to…):

By the end of Sprint 3, users will be able to receive their nutrition breakdown in under 10 seconds and be able to view a history of all their previous scans along with their accompanying AI breakdowns in a separate tab.

What will NOT be included this Sprint (scope constraint):

We will not be making major UI changes aside from the bare minimum to complete the sprint goal. We will also not be adding login functionality in this sprint so scans will not be tied to a specific account.

Section 3 - Backlog Selection (2–4 items MAX)

Priority	Backlog Item	Reason it supports Sprint Goal	Why this is realistic this Sprint
High	**Diagnose and fix the AI taking too long problem.**	**We make the AI analysis less than 10 seconds which is a core part of our sprint goal.**	**We already have the code implemented and it's just refinement of the AI processes.**
High	**Add scan history.**	**Another core part of our sprint goal is to add past scans so the user than see them.**	**We already have everything loaded after the user scans, so all we need to do is just save the object and make it editable.**
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.

Section 4 - Task Decomposition (All tasks ≤ 1 class period)

If you need less rows than provided just leave them blank. If you need more rows than provided consider if you can refactor tasks or if you have tasks that are outside of your current scope.

Task	Owner	Related Backlog Item	Done = (clear completion criteria)	Includes Testing?
Diagnose where the AI delay is coming from (prompt size,	**Siddharth**	**Fix AI Speed**	**Root cause is documented and shared with team**	**No**

API latency, response parsing, etc.)				
Refine the prompt to reduce token count without losing output quality	**Siddharth**	**AI Speed Fix**	**AI response is under 10 seconds consistently on 3+ different products**	**Yes**
Add async/loading state so the UI doesn't freeze while waiting for AI response	**Arda**	**AI Speed Fix**	**App shows a loading indicator and stays responsive during AI call**	**Yes**
Design and build the scan history UI screen (list of past scans with product name, date, and thumbnail)	**Arda**	**Scan History**	**Screen renders correctly with at least 2 saved entries visible**	**Yes**
Set up Azure storage structure for saving scan objects after each successful scan	**Varun**	**Scan History**	**Scan data is written to Azure and retrievable after app restart**	**Yes**
Hook up the scan result to automatically save to Azure on completion	**Varun**	**Scan History**	**Every completed scan appears in history without manual action from user**	**Yes**
Load and display saved scans from Azure on the history screen	**Leo**	**Scan History**	**History screen pulls real data from Azure and displays it correctly**	**Yes**
Allow user to tap a past scan	**Leo**	**Scan History**	**Tapping an entry opens the**	**Yes**

in history to view its full breakdown again			**full result screen with correct data**	
End to end test: scan a product, verify it saves to history, reopen app and confirm it persists	**Siddharth**	**Scan History**	**Data survives app close and reopen with no data loss or crashes**	**Yes**
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Choose an item.
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Choose an item.
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Choose an item.
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Choose an item.
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Choose an item.
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Choose an item.

Section 5 - Testing Plan (Must occur throughout Sprint)

Which tasks include testing and how:

For all of the tasks involving the new database integration we will need to test for correct read/write operations to ensure past scanned products are saving accurately. For the tasks focused on optimizing the OpenAI and OpenFoodFacts API calls we will include timing tests to reduce response time/latency. We will also test on both the school network and a hotspot to ensure the Azure deployment continues to work even on school Wi-Fi.

Who is responsible for testing:

The task owner is responsible for testing his commit, and the assigned Pull Request reviewer will do some secondary testing and verification before the code is merged.

How testing ensures Definition of Done is met before marking Done:

Since we have two reviewers/testers who will test thoroughly to make sure that the code runs correctly and the feature can be demonstrated to meet our definition of done, we can make

sure that no broken code or unresolved dependencies make it into the main branch prior to the sprint review.

Section 6 - Workflow & Board Discipline

When will the board be updated each day:

Within the first 5 minutes of our 50 minute class periods. This way we can make sure that everything is aligned and updated before we begin working each day.

How tasks will move (To Do → In Progress → Done):

Tasks will be moved to In Progress when a team member begins working on it on a dedicated feature branch. A task will move to a review stage when a PR is created. It only moves to Done after the PR is reviewed by another team member, approved, fully meets our definition of done, and merged into the main branch.

Rule to prevent last-minute work completion:

No new tasks can be moved to In Progress within 48 hours of the Sprint Review. Also, all Pull Requests must be submitted at least 24 hours before the review date so that the other team members have adequate time for peer review and merging.

Section 7 - Dependencies & Parallel Work Plan

Identify one key dependency:

The scan history feature depends on having a working local storage or database solution to persist scan records and their AI breakdowns across sessions. If we cannot get storage working reliably, the history tab has nothing to display.

What work can happen in parallel:

Performance optimization for getting the nutrition breakdown under 10 seconds and building the history tab UI can happen at the same time since they touch different parts of the app. One person can work on reducing API response time and caching while another builds the history screen layout and navigation.

What work must wait and why:

Populating the history tab with real data must wait until the storage solution is implemented and tested. We can build the UI first with placeholder data but connecting it to actual saved scans requires the database layer to be in place.

Section 8 - Risk & Adjustment Plan

Primary risk to completing Sprint Goal:

The 10-second performance target may be difficult to hit consistently because it depends on network speed, the Open Food Facts API response time, and the AI processing time. These are all factors outside our direct control.

Adjustment trigger (If _____ happens by _____, we will _____):

If we cannot consistently get responses under 10 seconds by the midpoint of the sprint, we will implement a loading indicator with partial results, so the user sees product info first while the AI breakdown loads in. That way the experience still feels responsive even if the full result takes longer.

Section 9 – Team Commitment

☒We confirm this plan is realistic and aligned to our Sprint Goal.

☒All tasks meet the ≤1 class period requirement.

☒Testing is integrated into our workflow (not delayed).